Architecture

Technology Stack

- **Jsonnet** + **Grafonnet** for dashboards
- **Jsonnet** for Mimir alert rules and Alertmanager config
- **Ruby** + **RSpec** for integration tests
- **JsonnetUnit** for unit tests
- **mimirtool** for alert rule deployment
- **asdf/mise** for tool version management

Key Directories

Directory	Purpose
`dashboards/`	Grafana dashboard definitions (Jsonnet -> JSON -> Grafana API)
`mimir-rules-jsonnet/`	Alert rule sources (Jsonnet) — compiled to `mimir-rules/`
`mimir-rules/`	**Auto-generated** YAML alert rules per tenant (do not edit directly)
`libsonnet/`	Shared Jsonnet libraries
`libsonnet/alerts/`	Alert processing library (adds Grafana links, enforces conventions)
`alertmanager/`	Alertmanager routing config (Jsonnet -> YAML -> K8s secret)
`metrics-catalog/`	Service definitions and SLI catalog (drives auto-generated alerts)
`mixins-monitoring/`	Reusable monitoring mixins (dashboards + rules + alerts)
`scripts/`	Build/generate/validate scripts
`docs/`	Runbook documentation
`on-call/`	On-call checklists
`spec/`	RSpec integration tests
`bin/`	Helper scripts (accessed via `glsh` CLI)

Dashboard Lifecycle

Dashboards are Grafana dashboards managed as code using Grafonnet. Each team can own a subfolder under `dashboards/` (e.g., `dashboards/dx/` for Developer Experience). See `dashboards/AGENTS.md` for the detailed development guide.

1. **Author**: `dashboards/<folder>/<name>.dashboard.jsonnet` using Grafonnet
2. **Generate**: `make generate` (runs `scripts/generate.sh`)
3. **Test**: `dashboards/upload.sh -D` (dry run) + `make validate-service-dashboards`

4. **Deploy**: CI uploads to https://dashboards.gitlab.net via Grafana API on main branch (gitlab.com only)
5. **API token**: From Vault at `${VAULT_SECRETS_PATH}/ops/grafana/api_token`

Alert Rule Lifecycle

1. **Author**: Edit Jsonnet in `mimir-rules-jsonnet/` or `libsonnet/alerts/`
2. **Generate**: `make generate` compiles Jsonnet to YAML in `mimir-rules/<tenant>/autogenerated-*.ym`
3. **Validate**: `mimirtool rules check` per tenant
4. **Deploy**: `mimirtool rules sync` per tenant on ops.gitlab.net main branch

Mimir tenants: `gitlab-gprd` (prod), `gitlab-gstg` (staging), `gitlab-ops`, `gitlab-pre`, `gitlab-observability`, `metamonitoring`, `runway`, etc.

Alertmanager Routing

1. **Author**: `alertmanager/alertmanager.jsonnet` (routing rules, receivers)
2. **Generate**: `./alertmanager/generate.sh` -> `alertmanager.yml`
3. **Test**: `make test-alertmanager` (amtool check + routing tests)
4. **Deploy**: K8s secret consumed by Prometheus Operator (on ops.gitlab.net)

Receivers include: Slack channels (#alerts, #production, #alerts-nonprod), Dead Man's Snitch, incident.io, Slackline bridges.

Metrics Catalog

Service definitions in `metrics-catalog/services/*.jsonnet` declaratively define services and their SLIs. Alert rules are auto-generated from these using multi-window multi-burn-rate (MWMBR) alerting via `libsonnet/alerts/service-component-alerts.libsonnet`.

Grafana Instances

- **GitLab-wide Grafana**: https://dashboards.gitlab.net — the primary Grafana instance for all GitLab teams. Dashboards from this repo deploy here.
- **DevEx Grafana**: https://dashboards.devex.gitlab.net — separate instance historically used by Developer Experience team. Being migrated to the GitLab-wide instance (see migration epic).

Known Patterns

- All alert rules include `grafana_dashboard_link` annotations that deep-link to the relevant Grafana dashboard

- Alert labels: `alert_type` (symptom/cause), `severity` (s1-s4), `rules_domain`, service identifiers
- ClickHouse is the primary data source for DX dashboards (not Prometheus)
- Dashboards use Grafana template variables: `$environment`, `$stage`, `$PROMETHEUS_DS`